

INT531

Site Reliability Engineering

Observability I · metrics and Prometheus

Week 4 · this afternoon we prepare the containment build

School of Information Technology, King Mongkut's University of Technology Thonburi

Lecture 2 h + Lab 1.5 h | Content updated 2026

scrape_interval: 15s

metrics



What we cover today

Lecture 2 h · lab 1.5 h — metrics in the morning, the parts count in the afternoon

01

Monitoring vs observability

How they differ, and why metrics alone are not enough

02

Metric types

Counter, gauge, histogram, summary — and choosing between them

03

Prometheus, the TSDB and Alertmanager

The whole stack, the pull model, and TSDB internals

04

PromQL and the cardinality trap

Computing an SLI from raw metrics, and what must never be a label

05

Install, configure targets, and test

Eight install steps, prometheus.yml, and six acceptance checks

06

Lab 4 — preparing the build

Count what exists, compute the BOM, check capacity before installing

Session learning outcomes

By the end of this session you will be able to...

SLO 4.1	Explain how monitoring and observability differ, and the limits of metrics	CLO3
SLO 4.2	Choose the metric type that suits what you need to measure	CLO3
SLO 4.3	Explain Prometheus's pull model and what it means for detecting dead targets	CLO3
SLO 4.4	Write PromQL for an availability SLI and a latency percentile	CLO2, CLO3
SLO 4.5	Estimate a metric's cardinality and avoid labels that take the system down	CLO3, CLO5
SLO 4.6	Install and configure Prometheus, Alertmanager and Grafana, and verify against the checks	CLO3, CLO4
SLO 4.7	Compute a parts list from per-unit rules and verify capacity before installing	CLO5, CLO7



01

Seeing it before the users tell you

Last week we diagnosed by hand; today we fit permanent instruments

How monitoring and observability differ

Not synonyms, and neither replaces the other

Aspect	Monitoring	Observability
What it answers	Things you predicted could break (known-unknowns)	Things you never anticipated (unknown-unknowns)
How it is set up	Decide in advance what to measure and when to alert	Collect enough detail to ask a new question later
Example question	Is CPU above 80% yet?	Why are only mobile users at 09:00 slow?
Main cost	Configuring and maintaining alerts	Data volume and cardinality

Metrics are cheap and excellent at answering prepared questions, and useless for unprepared ones — which is why logs and traces follow in week 5

Metric types, and choosing between them

Pick the wrong type up front and the calculation you want later becomes impossible

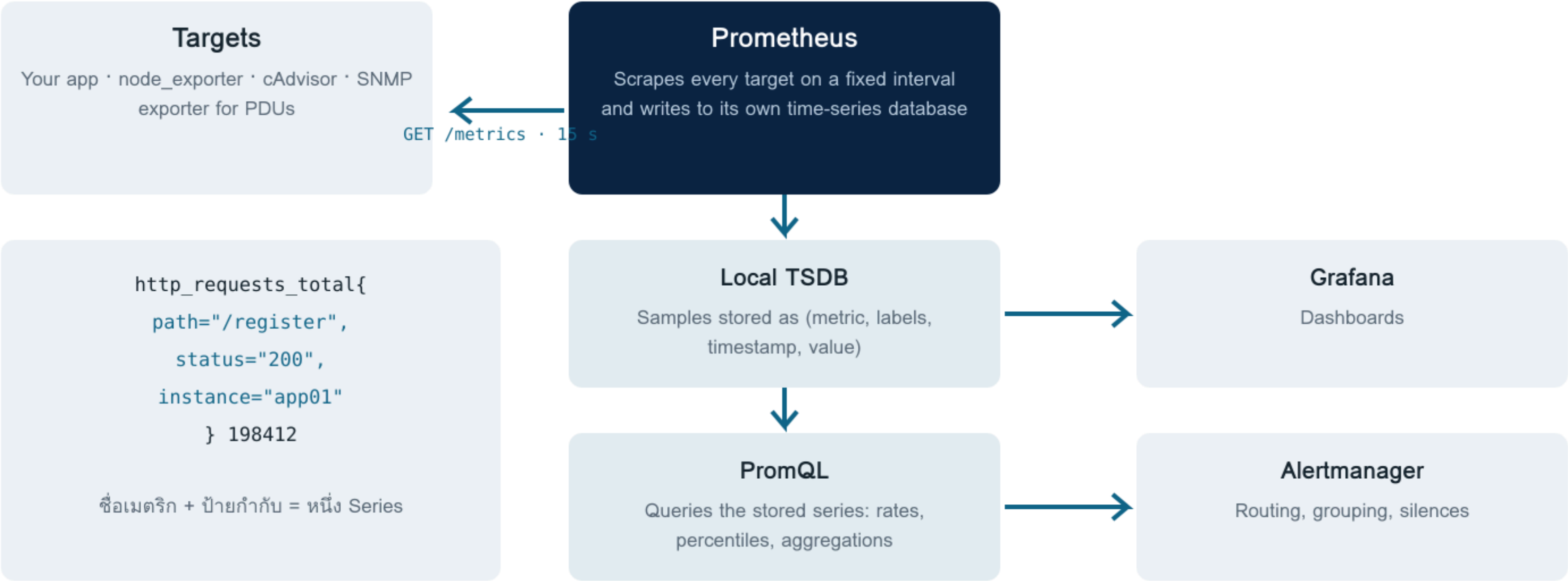
Type	Behaviour	Use it for	Example name
Counter	Only ever increases; resets to zero on restart	Cumulative counts such as requests or errors	http_requests_total
Gauge	Moves up and down freely	A value at a point in time: memory, temperature	node_memory_available_bytes
Histogram	Counts observations into buckets	Distributions such as latency; percentiles can be derived	http_request_duration_seconds_bucket
Summary	Computes quantiles in the application itself	When you never need to aggregate across hosts — you cannot	rpc_duration_seconds

A rule that holds almost always: if you want percentiles, use a histogram, because it aggregates across hosts. A summary does not.

Prometheus architecture

Understand the pull model and you understand why the up metric matters so much

Prometheus pulls; it is never pushed to



Because Prometheus pulls, a target that dies simply stops answering — and the up metric goes to 0. That alone tells you something is wrong.

The components of the Prometheus stack

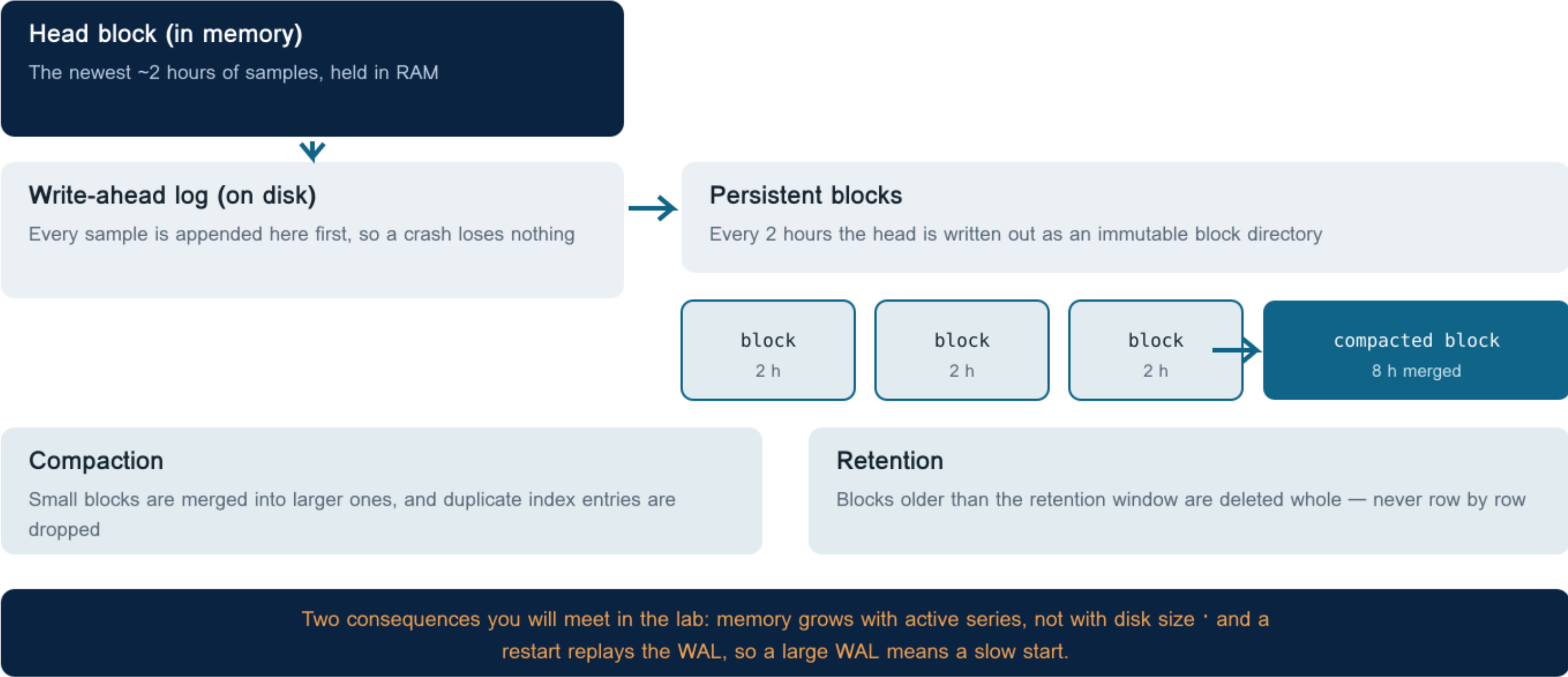
Five pieces, with responsibilities that do not overlap

Component	Responsibility	Default port	What breaks without it
Prometheus server	Scrapes targets, stores to the TSDB, evaluates rules	9090	No data and no alerts at all
Exporter	Translates a system's data into a format Prometheus can read	9100 (node)	Devices that speak another protocol cannot be scraped
Alertmanager	Routes, groups and delivers notifications	9093	Alerts fire but nobody is told
Grafana	Visualisation and dashboards	3000	You are left querying through the Prometheus UI
Pushgateway	Accepts values from jobs too short-lived to be scraped	9091	Short batch jobs record nothing

A caution: Pushgateway is only for genuinely short-lived jobs. Using it as a substitute for scraping destroys the up metric as a liveness signal.

Inside the local TSDB

Knowing this explains why memory grows, and why a restart can be slow



PromQL — computing an SLI from raw metrics

These three cover most practical work

Requests per second

```
rate(http_requests_total[5m])
```

rate() takes the slope of a counter over a window and handles counter resets for you. Never subtract raw values yourself.

Success ratio (availability SLI)

```
sum(rate(http_requests_total{status!~"5.."}[5m]))  
/ sum(rate(http_requests_total[5m]))
```

Numerator: requests that were not server errors. Denominator: all requests. This is exactly the SLI equation from week 2.

Latency at p99

```
histogram_quantile(0.99,  
  sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
```

You must aggregate by the le label first. Forget sum by (le) and you get a per-instance figure, which is not what you asked for.

Notice the SLI we compute in PromQL is the same equation we wrote by hand in week 2 — only the thing doing the arithmetic has changed

The cardinality trap

The number-one cause of Prometheus falling over in real organisations

Cardinality: every label multiplies, it never adds

A sensible metric

path	12	×
method	4	×
status	6	×
instance	20	×

$$12 \times 4 \times 6 \times 20 = 5,760 \text{ series}$$

Fine on a single Prometheus

The same metric with one careless label

path	12	×
method	4	×
status	6	×
instance	20	×
user_id	3,200	×

$$12 \times 4 \times 6 \times 20 \times 3,200 = 18,432,000 \text{ series}$$

Will take the server down

Rule: a label may only hold values from a small, closed set. User IDs, request IDs, email addresses and full URLs are never labels — they belong in logs and traces.

The rest of the PromQL you will actually need

Result types, selectors, and the functions that come up daily

What to know	Form	When to use it
Instant vector	<code>http_requests_total</code>	One latest value per series — current state
Range vector	<code>http_requests_total[5m]</code>	A window of values — required by <code>rate()</code> and <code>increase()</code>
Label selectors	<code>{job="api", status=~"5.."} </code>	<code>=</code> exact, <code>!=</code> not equal, <code>=~</code> regex match, <code>!~</code> regex mismatch
Aggregation	<code>sum by (path) (...)</code>	Combine series keeping the named labels; <code>without</code> keeps all but those named
<code>increase()</code>	<code>increase(errors_total[1h])</code>	How much a counter grew over a window — how many times it happened
Operators	<code>a / b</code> , <code>a > 0.99</code> , <code>a and b</code>	Ratios, comparisons against a target, and alert conditions

Common traps: using `rate()` on a gauge (it is for counters only), and choosing a window in `[]` shorter than four scrape intervals.

Worked example: how much disk does that need?

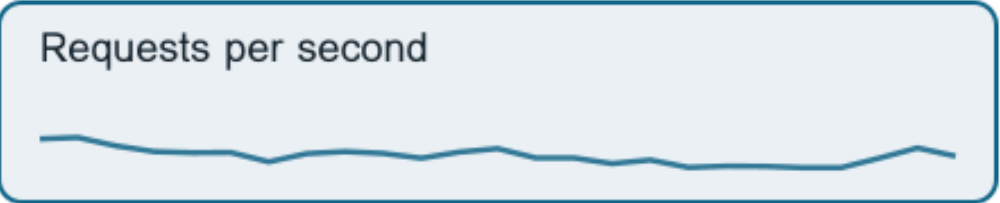
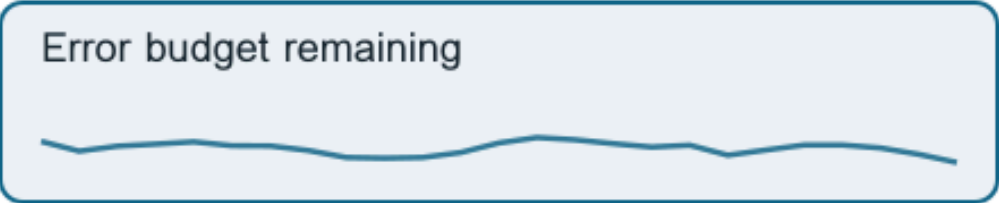
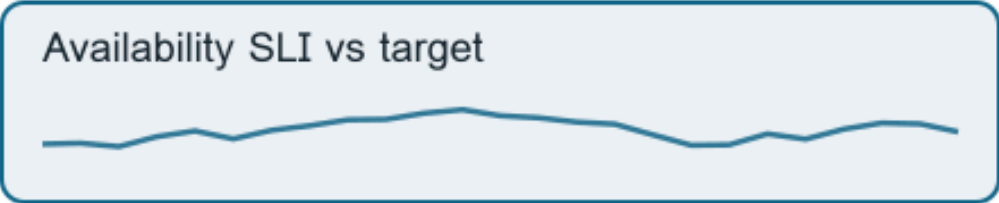
The same numbers from the cardinality slide, carried through to the disk you must provision

storage ≈ series ÷ scrape_interval × bytes_per_sample × retention_seconds		
Active series	5,760	from the cardinality slide
Scrape interval	15 s	one sample per series per scrape
Samples per second	384	5,760 ÷ 15
Bytes per sample	≈ 2 B	after compression, a common rule of thumb
Per day	≈ 66 MB	384 × 86,400 × 2
Retention 15 days	≈ 1.0 GB	well within one small server
Now redo it with the user_id label: 18,432,000 series → 1.23 M samples/s → about 212 GB per day		

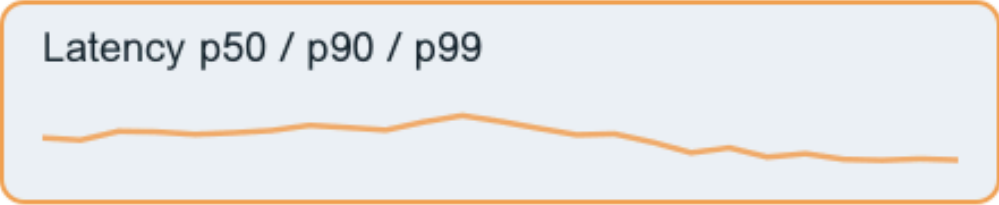
Designing a dashboard that answers questions in order

A good dashboard reads top to bottom without anyone narrating it

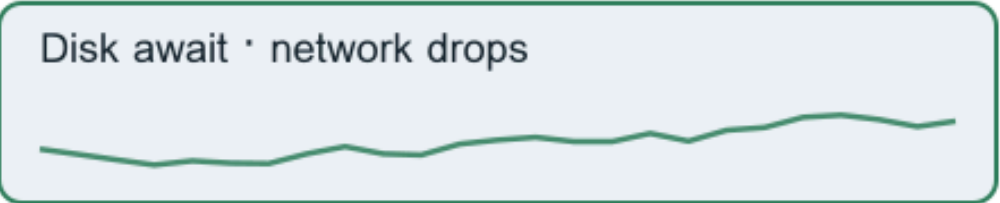
Row 1 — is the service healthy?



Row 2 — how bad, and for whom?



Row 3 — which resource is hurting?



Top row visible without scrolling

One question per panel

Same time range on every panel

Read top to bottom: is it broken * how badly * what is causing it. A dashboard that needs explaining is not finished.

Case study 1 — when the monitoring system took itself down

A sequence that repeats across organisations and ends the same way almost every time

Day 0 · 14:20

A debugging label is added

An engineer adds `order_id` to a metric to chase a bug. It ships that afternoon.

Day 0 · 16:40

Series count starts climbing

Active series go from 40 k to 900 k within two hours. Nobody is watching that number.

Day 1 · 02:10

Prometheus is OOM-killed

Memory exhausted. The process restarts, replays its write-ahead log, and dies again.

Day 1 · 02:10

Alerting goes silent

No metrics means no alerts. A real database incident at 03:00 goes unnoticed for 50 minutes.

Day 1 · 09:30

Recovery

The label is dropped, the TSDB is trimmed, and a series-count alert is finally added.

Three lessons

- 1 The monitoring system needs monitoring — alert on active series count and on Prometheus memory
- 2 Any new label needs a review, because a label is a cardinality decision, not a formatting choice
- 3 Losing observability is itself an incident, even when the service is fine

The data centre has metrics too

Physical infrastructure can carry metrics and SLOs like anything else

What we measure	Metric type	Collected from	Decides what
PDU load per phase (A)	Gauge	SNMP exporter on the PDU	Whether there is headroom to install more servers
Rack inlet and outlet temperature	Gauge	In-rack sensors over SNMP	Whether the containment is actually working
PSU status per device	Gauge (0/1)	IPMI or Redfish exporter	Whether anything is running on a single PSU
Switch ports in use	Gauge	SNMP exporter on the ToR	When another switch has to be ordered

All four are numbers your group needs this afternoon. The only difference is that today you count them by hand, and next week the system counts them for you

Grafana — from data to something you can decide on

Four things to get right at the start

Data source

Point at Prometheus by its network name, `http://prometheus:9090`, not `localhost` — they are different containers

Panels and queries

One panel answers one question. Title the panel with the question, not the metric name, and always set the unit

Variables

Use `$instance` or `$job` so one dashboard serves every host, instead of copying dashboards per machine

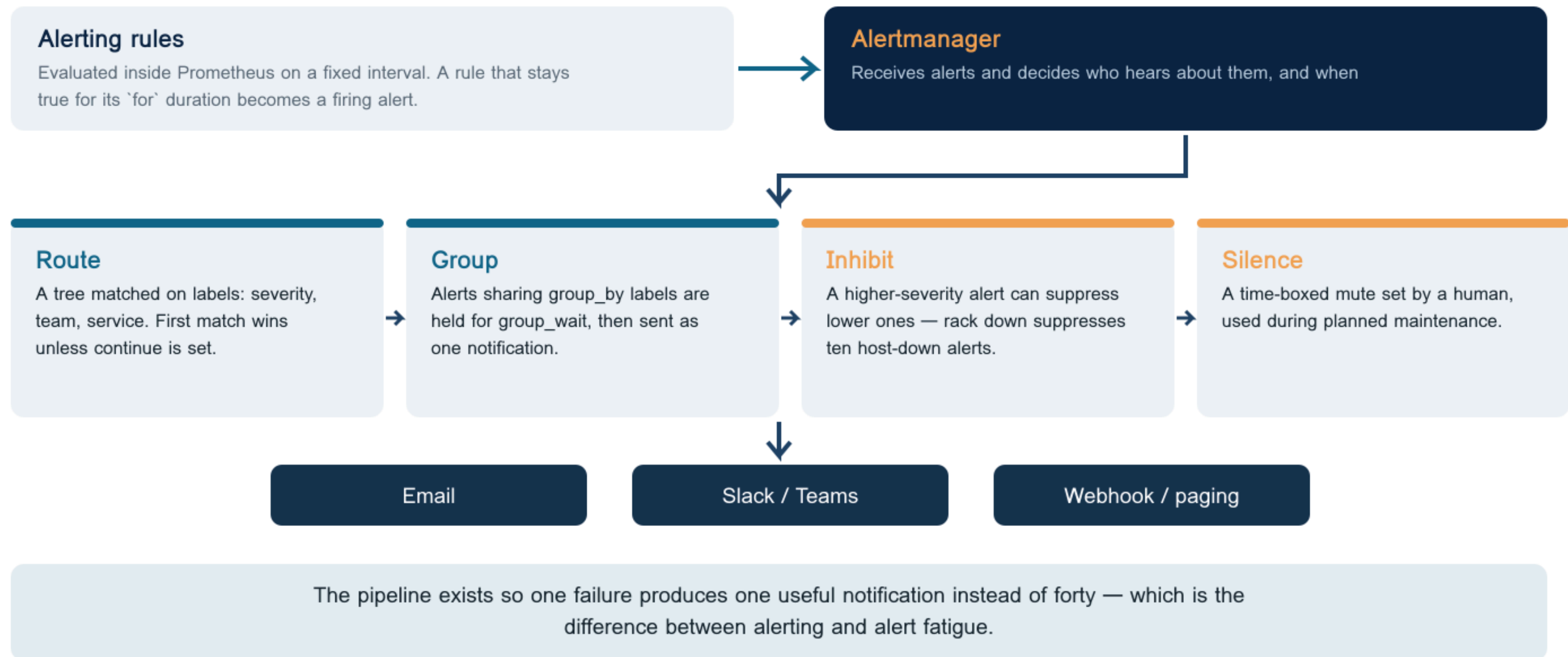
Provisioning

Keep dashboard JSON in Git and let Grafana load it at start, so the dashboard can be rebuilt identically

A dashboard built by hand and never exported is a dashboard that disappears with the machine it lived on

Alertmanager — from a rule to the person who must know

The pipeline that turns one failure into one message



Installation, in this order

Eight steps, about 30 minutes, provided none is skipped

- 1 Lay out the directories** Create monitoring/ with prometheus/, alertmanager/ and grafana/ inside, so volume mounts stay obvious
- 2 Write docker-compose.yml** Four services — prometheus, alertmanager, grafana, node-exporter — on one network
- 3 Write prometheus.yml** Set global.scrape_interval and a scrape_configs entry for every target
- 4 Validate before running** Run promtool check config prometheus.yml and get a clean pass first
- 5 Bring it up** docker compose up -d, then confirm every container is running, not restarting
- 6 Check the targets page** Open <http://localhost:9090/targets> — everything must read UP; if not, read the Error column
- 7 Test a query** Run up, then a rate(...) query, and get real results before building any dashboard
- 8 Connect Grafana** Add the data source and build the first panel from a query you already proved works

Configuring targets in prometheus.yml

The actual file shape used in the lab

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - /etc/prometheus/rules/*.yaml

alerting:
  alertmanagers:
    - static_configs:
      - targets: ['alertmanager:9093']

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'node'
    static_configs:
      - targets: ['node-exporter:9100']
      labels:
        rack: 'CT-01'
        room: 'CB-204'
```

job_name becomes a label on every metric

Name it meaningfully — it turns into the job label you will filter on in every later query

Labels set here attach to every series

Use them for fixed context such as rack or room, which is what links metrics to the physical room

Hostnames must be service names

Under Docker Compose use node-exporter:9100, not localhost:9100 — different containers

Watch how many labels you add

Every label here multiplies against existing cardinality, so add only what you will actually query

Testing before you call it done

All six must pass for the install to count as successful



The config is valid

`promtool check config prometheus.yml`

Must report SUCCESS with no errors



Every target is scraped

Open `/targets`

Every row UP, last scrape within one interval



Metrics have real values

Query: `up`

Returns 1 for every target



Rates compute

Query `rate(node_cpu_seconds_total[5m])`

Returns results, not an empty set



Rules are loaded

Open `/rules`

Your rules appear and none is in error



Alerts reach the destination

Fire a test alert into Alertmanager

You see the message at the receiver, not just in the UI

The last one is the most commonly skipped, and the only one that proves the notification path works end to end



02

Lab 4 — preparing the containment build

The old rack is empty · today we count, compute, and check whether the plan actually fits

Where the project stands

End of week 3 · start of week 4

Done

Everything is out of the old rack, recorded in INV-01 and MED-01, staged and sorted by destination

Today

Count what actually exists, compute what has to be procured, and check whether space and power are sufficient

Not started

Installation into the containment rack begins only once the BOM checks out and the open items are closed

The constraint to remember

About ten more servers arrive in phases with no confirmed dates, so the install plan must tolerate partial delivery

Seeing the room in five layers before planning the install

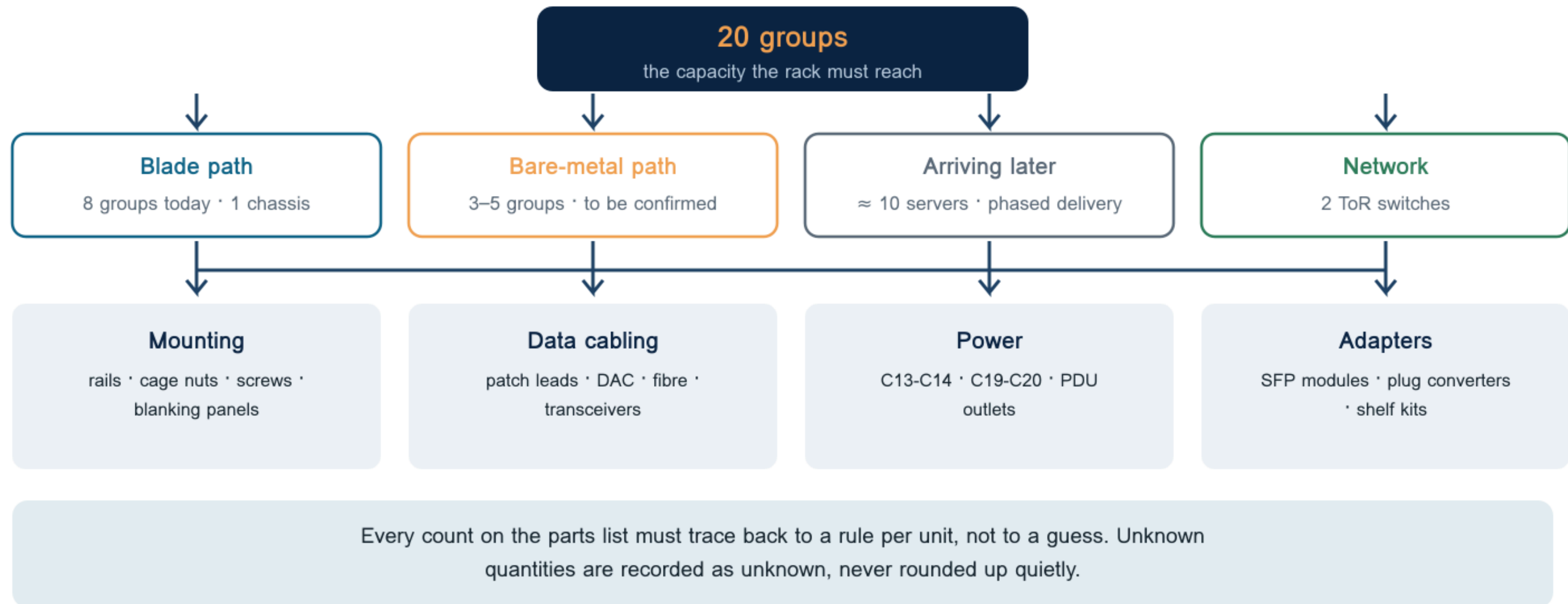
Use this as the frame for the group discussion this afternoon

The question this layer must answer	
5 • Monitoring Sensors, PDU polling, PUE	Which numbers will we read, and from where?
4 • Network ToR switches, patch panel, uplinks to core	Port type, cable lengths, how many ports left spare?
3 • Power Feeds, UPS, PDU A and PDU B, outlets	Load per phase, what survives one PDU failing, inrush at power-on?
2 • Racks and airflow Containment rack, U layout, blanking panels	What sits where, and is every gap capped?
1 • Floor and routing Floor loading, cable trays, door and aisle widths	Can the equipment physically get there and be carried in?
Plan top down, build bottom up Decisions flow from monitoring back to the floor, but installation starts at the floor and works up. A layer that was never decided becomes a layer that gets improvised on the day.	

Turning “20 groups” into a parts list

Every quantity comes from a per-unit rule, never from a guess

What 20 groups actually turns into on the parts list



The per-unit rules

Learn the rules, not the numbers — the numbers change as counts get confirmed

Item	Per-unit rule	Where the rule comes from
Rail kits	1 set per device mounted in the rack	Chassis, bare-metal servers and ToR switches all count
Cage nuts and screws	8 of each per device	Four per post, two posts
Blanking panels	Equal to the U left empty after everything is installed	Every gap must be capped or hot air recirculates
Power cords	1 per PSU	Dual-PSU devices split across PDU A and PDU B
Data cables	1 per port in use	Bare metal: 2 data ports plus 1 management port
Cable labels	2 per cable, of every type	Both ends must be labelled under control A.7.12
SFP+ modules	2 per fibre run	One at the ToR end and one at the core end

The first pass says it does not fit — and that is the right outcome

What the BOM file returns with today's starting numbers

Groups supported	22 (8 blade + 4 bare + 10 arriving)	Pass — two above target
Rack units required	44U in a 42U rack	Fail — 2U short
Total power draw	9,200 W	Fail — above the 2,944 W usable on a single PDU
PDU outlets required	36 of 48 available	Pass

Failing on paper is far better than discovering it with a server already halfway into the rails. Today's job is choosing a remedy, not massaging the numbers until they pass

Case study 2 — everything powered on at once, and the breaker tripped

The same mistake in the physical world, and it bears directly on this afternoon

Steady-state draw

9,200 W

what the BOM calculation predicted

Inrush at power-on

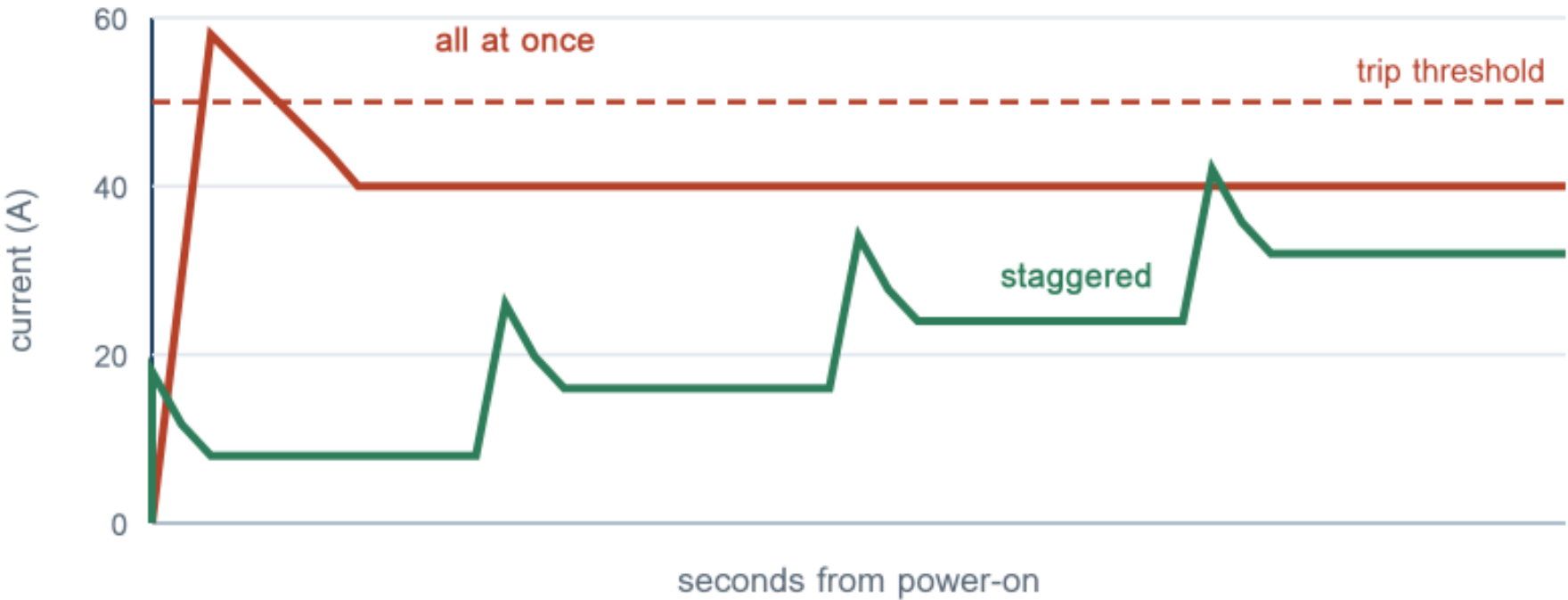
up to 3–5×

for a few hundred milliseconds per PSU

Breaker rating

16 A

trips on the combined surge, not the steady load



Nameplate figures describe steady state.
Startup behaviour is a different number, and
it is the one that trips breakers.

The fix costs nothing: power on in groups of four, thirty seconds apart. The lesson costs a lot if you learn it at 02:00 with users waiting.

Worked example: what happens once the numbers are confirmed

Suppose the recount finds five bare-metal groups, and the chassis holds only six blades

Item	Before confirming	After confirming	Consequence
Bare-metal groups	4 (estimated)	5 (counted)	One more set of rails, power cords and DAC cables
Chassis slots	8	6	Eight blade groups now need two chassis, not one
Chassis count	1	2	Another 10U of space and four more PSUs
Rack units required	44U	56U	Far beyond the 42U rack — a second rack or a phased install
Total power draw	9,200 W	12,800 W	Further above the PDU rating; additional circuits are now certain

Two unconfirmed numbers changed the entire plan — which is exactly why the open items close before anything is ordered

Options when space or power falls short

Each has a cost; the group must choose and justify

If rack units fall short

- Specify 1U rather than 2U for the servers still to be procured
- Install in phases, fitting what the space allows for now
- Request a second containment rack
- Revisit whether the 4U reserved for patching can be reduced

If power falls short

- Add circuits and PDUs to match the computed load
- Move to higher-rated three-phase PDUs
- Stagger power-on to limit inrush (helps only at startup)
- Re-check nameplate figures, since the current values are estimates

Every option goes through a change request (CR-01) stating its impact, under control A.8.32

Open items — close these before ordering

All eight live on the open-items sheet of the BOM file

- | | | |
|---|--|--|
| 1 | The exact number of bare-metal groups (3, 4 or 5) | Drives rails, power cords, DAC cables and rack units directly |
| 2 | Blade chassis spec and slot count | If it holds fewer than 8 blades you need two chassis, which changes everything |
| 3 | Delivery dates for the ~10 incoming servers | Determines whether this is one install or several |
| 4 | ToR switch port type (SFP+ or RJ45) | Determines DAC versus Cat6A, and whether modules are needed |
| 5 | Real nameplate power for every device | Current values are estimates; if reality is higher the overrun grows |
| 6 | Rack depth and post hole type | Rail kits must match the rack or they simply will not mount |
| 7 | Free PDU outlets actually available | If short, another PDU is needed, which changes the load calculation |
| 8 | Distance from rack to patch panel and core room | Sets the fibre lengths to order — measure the cable route, not the straight line |

Controls covering procurement and installation

In addition to those used in weeks 1–3

A.5.9

Inventory of assets

Every newly procured item is registered before it is installed

A.5.10

Acceptable use of assets

Items drawn from stores are used only for the stated work; surplus goes back

A.7.8

Equipment siting and protection

The U layout is approved before installation starts

A.7.11

Supporting utilities

Power and cooling are checked before anything is energised

A.7.12

Cabling security

Power and signal on separate trays, both ends labelled

A.8.32

Change management

Installation and any change of plan go through CR-01 with a rollback plan

What “done” looks like today

Checked before the session ends

☐

The group's BOM has the 'already have' column filled from a real count

Inventory lead

☐

The capacity sheet is complete and names which lines fail

Technical reviewer

☐

A remedy is chosen for every failing line, with reasoning

Change owner

☐

All eight open items have an owner and a closing date

Change owner

☐

Counted parts are stored properly and not mixed with other groups'

Safety officer

Assignments and next week

Week 5: Observability II — logs and distributed tracing

Group work, due in 5 days

- The completed BOM, with the capacity sheet passing on every line
- A record of the option chosen, with at least three lines of reasoning each
- Open items now closed, with evidence such as nameplate photographs
- A draft CR-01 for next week's installation

Individual, before the next session

- Read Observability Engineering, chapters 1–3
- Complete pre-class quiz 4 on the LMS
- Get Prometheus and Grafana running; submit a screenshot of the targets page showing up = 1
- Write the PromQL for your sample app's availability SLI as a single line

Prepare for next week

- Expose a /metrics endpoint that Prometheus can scrape
- Add at least one custom metric to the app
- Bring a cardinality question about a metric you created

The question that opens next session: metrics can tell you 1% of requests failed, but not whose requests or at which step — what else do we need to collect?

References

Split by part of the session

For the lecture

- Beyer, B. et al. (2016). Site Reliability Engineering. O'Reilly. Ch. 6 (Monitoring Distributed Systems)
- Majors, C., Fong-Jones, L., & Miranda, G. (2022). Observability Engineering. O'Reilly. Ch. 1–3
- Prometheus documentation — metric types, querying basics, and naming/label best practices
- Grafana Labs — dashboard best practices

For the field work

- ISO/IEC 27001:2022 — Annex A: A.5.9, A.5.10, A.7.8, A.7.11, A.7.12, A.8.32
- ISO/IEC 22237 series — data centre facilities and infrastructures (power distribution and layout parts)
- The manufacturer's installation guides for the rack and blade chassis actually in use
- The course forms pack — CR-01, INV-01, PWR-01, RACK-01, CAB-01

Today in three sentences

01

A metric is counting with a rule attached; choose the wrong type up front and the calculation you want later is simply unavailable

02

Labels multiply, they do not add — an unbounded label is how a monitoring system takes itself down

03

A parts list works the same way: every quantity traces to a per-unit rule, and failing on paper beats failing with a server in your hands

Week 5 · Observability II — when metrics say something is wrong but cannot say where

A note on the artwork (instructor slide)

Not for projection

Artwork in this deck

The title image, the Prometheus diagram, the cardinality panel and the BOM chart were drawn as vectors for this course and can be edited freely

If you want real photographs

Free sources cleared for commercial use: Unsplash, Pexels, Pixabay · Search terms: server rack installation, cable management, rail kit, data center staging

A caution under A.7.4

Do not publish photographs showing asset tags, network diagrams or screens with credentials outside the course